



Borislav Sabev - Borkata
CTO & Software Craftsman

Static Analysis & Automated Refactoring in the Era of AI-Assisted Coding

Why deterministic tooling is the guardrail that makes AI safe to ship

Партньори:



Следете актуалните обяви за **IT Management**

DEV.BG

This passed code review. Ship it?

```
/*
 * Repository contract (the part that matters):
 *   CustomerRepository::find(int $id): ?Customer
 *   OrderRepository::latestFor(Customer $c): ?Order // null → never ordered
 */
public function receiptline(int $customerId): string
{
    $customer = $this->customers->find($customerId);

    if ($customer === null) {
        throw new CustomerNotFound($customerId);
    }

    $order = $this->orders->latestFor($customer);

    return sprintf(
        '%s spent %s on %d items',
        $customer->name(),
        $order->total()->format(),
        count($order->items()),
    );
}
```

This passed code review. Ship it?

```
/*
 * Repository contract (the part that matters):
 *   CustomerRepository::find(int $id): ?Customer
 *   OrderRepository::latestFor(Customer $c): ?Order // null → never ordered
 */
public function receiptLine(int $customerId): string
{
    $customer = $this->customers->find($customerId);

    if ($customer === null) {
        throw new CustomerNotFound($customerId);
    }

    $order = $this->orders->latestFor($customer);

    return sprintf(
        '%s spent %s on %d items',
        $customer->name(),
        $order->total()->format(),
        count($order->items()),
    );
}
```

The deterministic verdict

```
```  
Line ReceiptService.php

36 Cannot call method total() on Order|null.
37 Cannot call method items() on Order|null.
[ERROR] Found 2 errors
```
```

Deterministic tools:

- provably correct
- repeatable
- no hallucinations

Probabilistic AI:

- fast
- fluent
- confidently wrong

- AI raised velocity but did nothing for trust
- Deterministic tooling is a valid check gate



Roadmap: The Trust Pipeline

AI made writing code cheap. It made ***trusting* code the bottleneck.**

- One repo
- 5 Layers (or maybe 6?)
- Each one makes AI-assisted code a little safer to ship



Style
PHPCS · CS-Fixer



Types
PHPStan



Transform
Rector



The Gate
CI / CD



Everywhere
Every stack



AI-Native
The payoff

Layer 1: Style & Consistency

Consistency used to be free. AI made it a problem again.

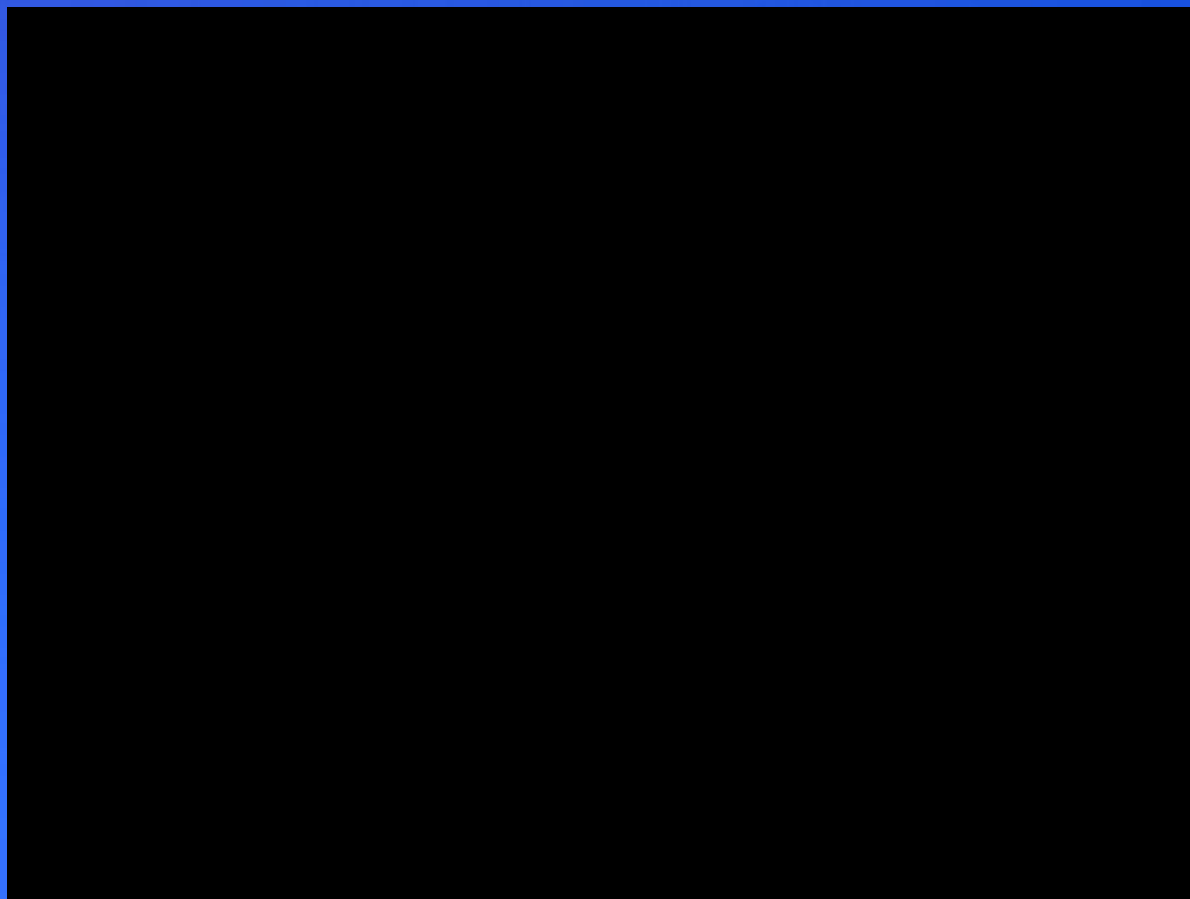
- Every generation drifts in style
- Style noise buries real review signal
- Table stakes - so we automate it and move on

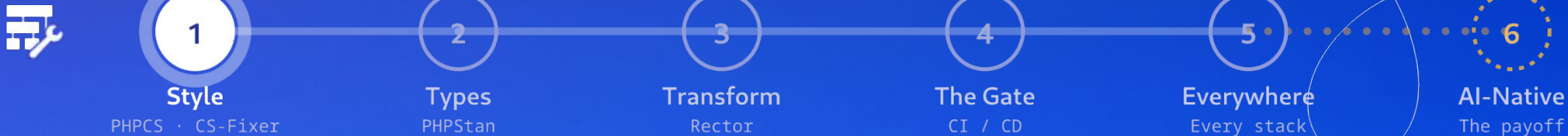
Tools in PHP

- PHP_CodeSniffer
- PHP-CS-Fixer

Demo: PHPCS -> PHP-CS-Fixer

No human touched this. No code review spent.





Layer 1: Takeaway

Style is the one thing AI conforms to well

- Hold that thought
- Give AI tooling the ruleset
- Mechanical problems deserve mechanical, deterministic solutions - not CRs



Style
PHPCS · CS-Fixer



Types
PHPStan



Transform
Rector



The Gate
CI / CD



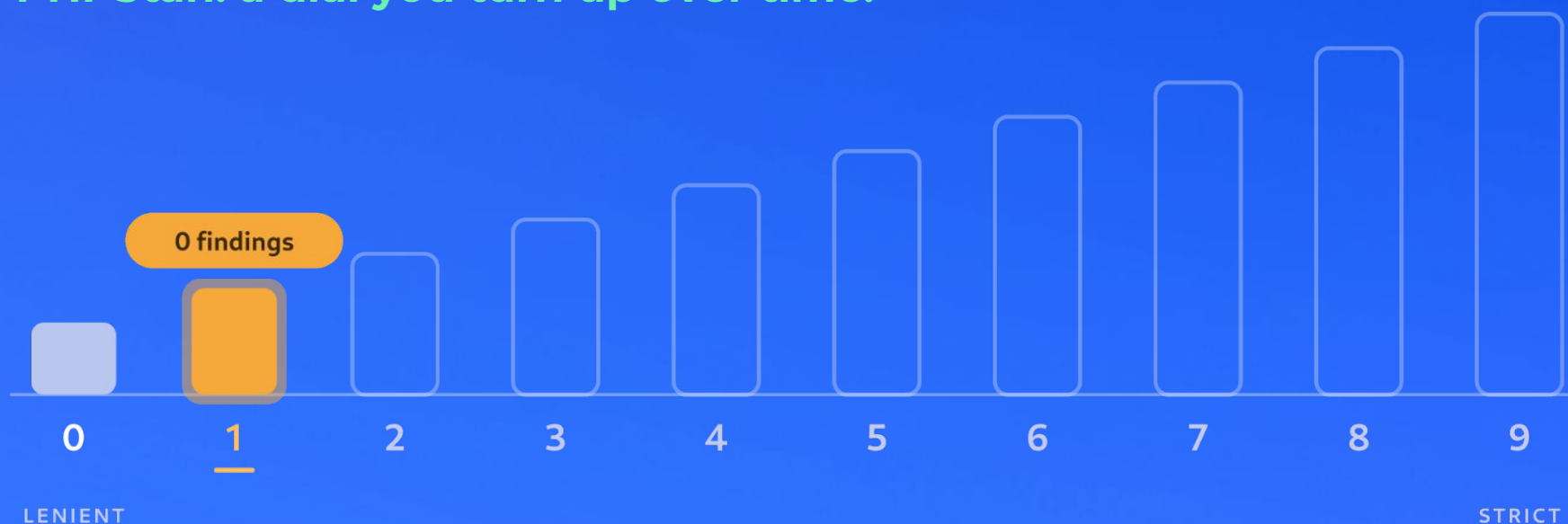
Everywhere
Every stack



AI-Native
The payoff

Layer 2: The dial

PHPStan: a dial you turn up over time.





Style
PHPCS · CS-Fixer



Types
PHPStan



Transform
Rector



The Gate
CI / CD



Everywhere
Every stack



AI-Native
The payoff

Layer 2: The dial

Same code. Turning up the dial.





Style
PHPCS · CS-Fixer



Types
PHPStan



Transform
Rector



The Gate
CI / CD



Everywhere
Every stack



AI-Native
The payoff

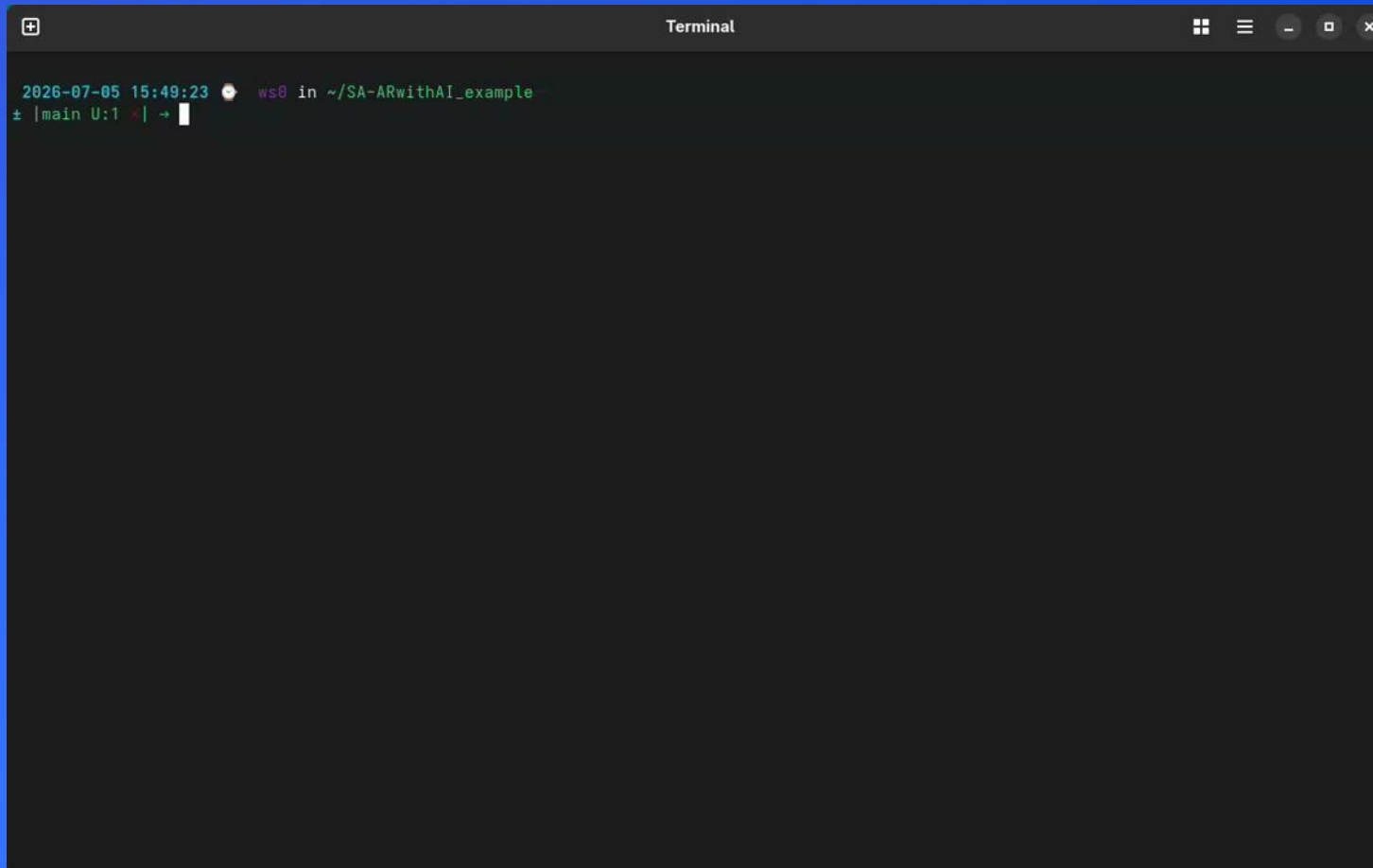
Layer 2: The dial

Same code again. Turn it up even more!



Demo: The PHPStan level walk

The test suite passes. Ship it?



```
Terminal
2026-07-05 15:49:23 ws0 in ~/SA-ARwithAI_example
± |main U:1 x| →
```

Demo: the three scary bugs

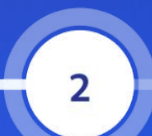
Every one of these passes review and then fatals at runtime

```
-----
Line  OrderService.php
-----
19    Method Demo\Service\OrderService::summary() has parameter $id with no
      type specified.
23    Call to an undefined method Demo\Model\Customer::getEmailAdress().
23    Cannot call method getCustomer() on Demo\Model\Order|null.
27    Method Demo\Service\OrderService::totalCents() has parameter $id with
      no type specified.
31    Cannot call method getTotal() on Demo\Model\Order|null.
31    Method Demo\Service\OrderService::totalCents() should return int but
      returns string.
35    Method Demo\Service\OrderService::reprice() has parameter $id with no
      type specified.
39    Cannot call method getTotal() on Demo\Model\Order|null.
51    Unreachable statement - code above always terminates.
-----
```

[ERROR] Found 9 errors



Style
PHPCS · CS-Fixer



Types
PHPStan



Transform
Rector



The Gate
CI / CD



Everywhere
Every stack



AI-Native
The payoff

Layer 2: Baselining

Inherited a 200k-line mess? `--generate-baseline` it!

1

Generate the baseline



baseline: 1,240

Snapshot every existing error into a baseline file — frozen.

2

Hold the line

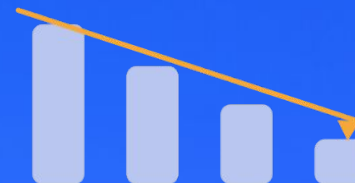


0 new errors

New code is held to the bar.
Zero new errors get merged.

3

Burn it down



1,240 → 0

Shrink the baseline over time.
The number only ever falls.

THE RATCHET · QUALITY ONLY GOES UP



Партньори:



Следете актуалните обяви за IT Management

DEV.BG



Style
PHPCS · CS-Fixer



Types
PHPStan



Transform
Rector



The Gate
CI / CD



Everywhere
Every stack

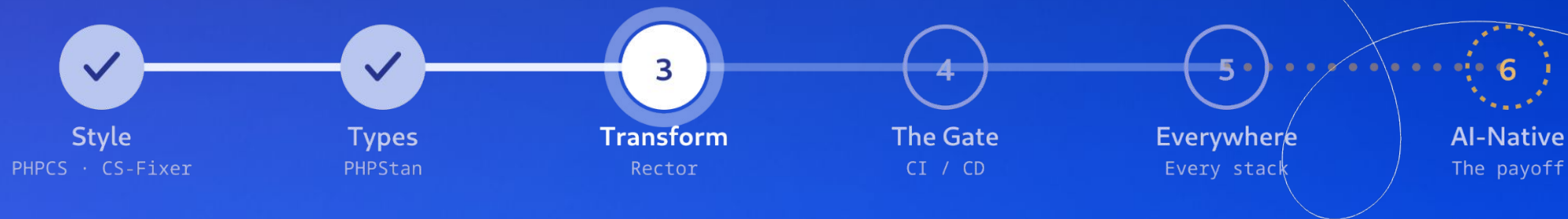


AI-Native
The payoff

Layer 2: Takeaway

The report card that AI doesn't get to argue with

```
"src/Service/OrderService.php": {  
  "errors": 9,  
  "messages": [  
    {  
      "message": "Call to an undefined method Demo\\Model\\Customer::getEmailAdress().",  
      "line": 23,  
      "ignorable": true,  
      "identifier": "method.notFound"  
    },  
    ....  
  ]  
}
```



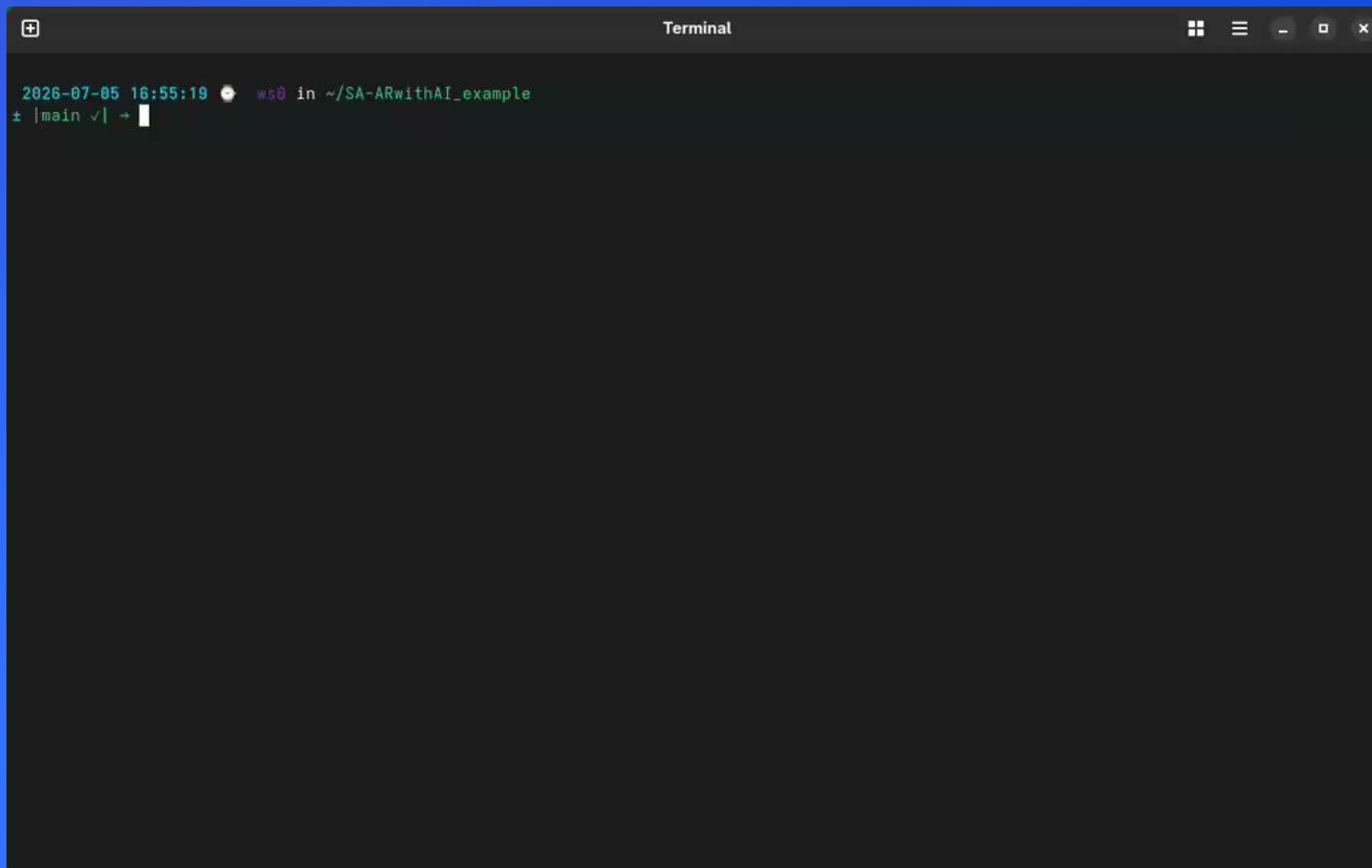
Layer 3: Automated Transformation

Stop flagging. Start fixing — at scale.

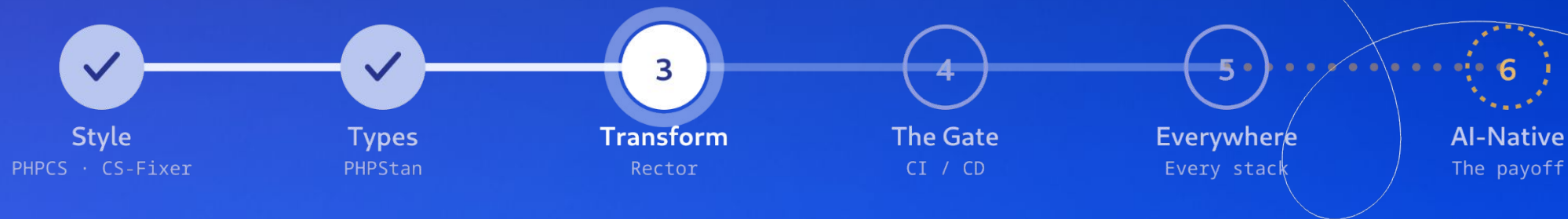
- PHPStan tells you what's wrong
- Rector changes it, identically, across every file

Demo: PHP 7.4 -> 8.3 modernization

On any real codebase - this would've been many days of manual PRs



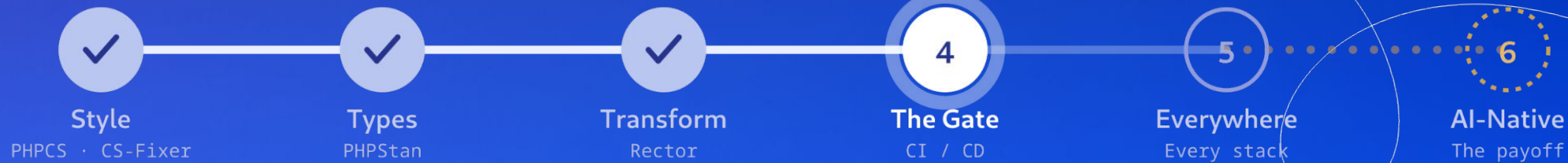
```
Terminal
2026-07-05 16:55:19 ws0 in ~/SA-ARwithAI_example
± |main ✓| →
```



Layer 3: Takeaway

Stop flagging. Start fixing — at scale.

- AI can do these refactors too
- Rector does them deterministically and identically across 10,000 files
- Tools like Rector are your automated refactoring pivot



Layer 4: The CI/CD Gate

None of this matters if a human has to remember to run it.

- Shift from “a bag of tools” -> “a system”
- Individual discipline does not scale. “The gate” does.

Demo: The CI/CD gate

Same pipeline, two commits. The gate decides.

```
41     return $order->getTotal()->cents() * 2;
42 }
43
44 + public function receiptTotal(int $id): int
45 + {
46 +     $order = $this->repository->find($id);
47 +
48 +     return $order->getTotal()->cents();
49
50 + }
51
52 + public function status(Order $order): string
53 + {
54 +     if ($order->isPaid()) {
```

▼  Check failure on line R48

Cannot call method getTotal() on Demo\Model\Order|null.

 CI / quality [View details](#)

▼  Check failure on line R48

Cannot call method getTotal() on Demo\Model\Order|null.

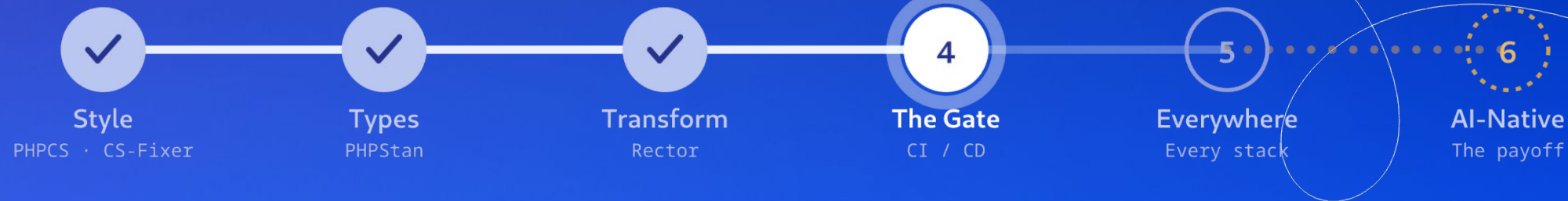
 CI / quality [View details](#)

 **All checks have passed**
2 successful checks

  CI / quality (pull_request) Successful in 17s

  CI / quality (push) Successful in 15s

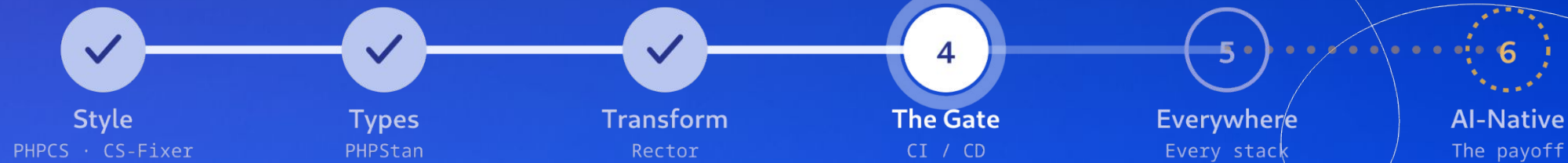
 **No conflicts with base branch**
Merging can be performed automatically.



Layer 4: Three patterns

Where trust becomes policy, not individual discipline

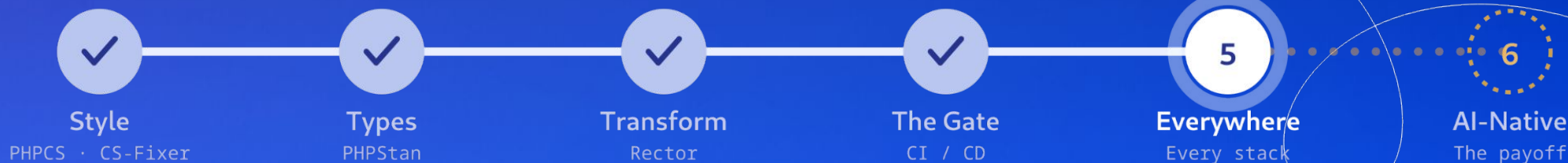
- Pre-commit vs. CI
- Fail-fast ordering
- Baseline-as-ratchet



Layer 4: Takeaway

A standard nobody can merge past beats a standard everybody agrees with

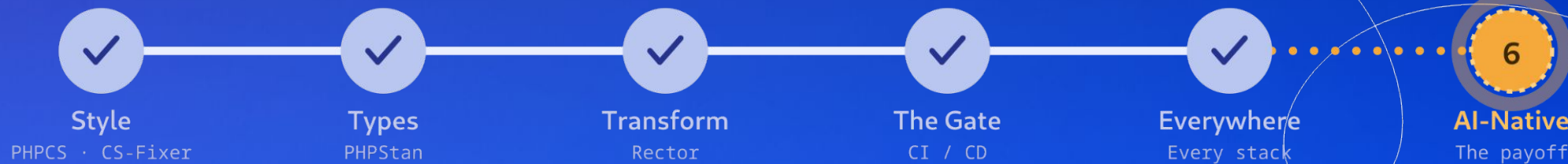
- Same bar for both humans and AI agents
- Fail-fast ordering
- Baseline-as-ratchet



Layer 5: 1 playbook $\rightarrow$ ∞ ecosystems

ECOSYSTEM	STYLE / FORMAT	CORRECTNESS / TYPES	TRANSFORMATION
PHP	PHPCS PHP-CS-Fixer	PHPStan, Psalm	Rector
JS / TS	ESLint, Prettier	tsc ESLint type rules	jscodeshift ts-morph
Python	Ruff, Black	mypy, Pyright	Bowler, LibCST
Java	Checkstyle, Spotless	SpotBugs, Error Prone, NullAway	OpenRewrite
.NET / C#	dotnet format, StyleCop	Roslyn analyzers + nullable refs	Roslyn analyzers + code fixes
Any language polyglot	EditorConfig, pre-commit	Semgrep, CodeQL	ast-grep, Semgrep --autofix

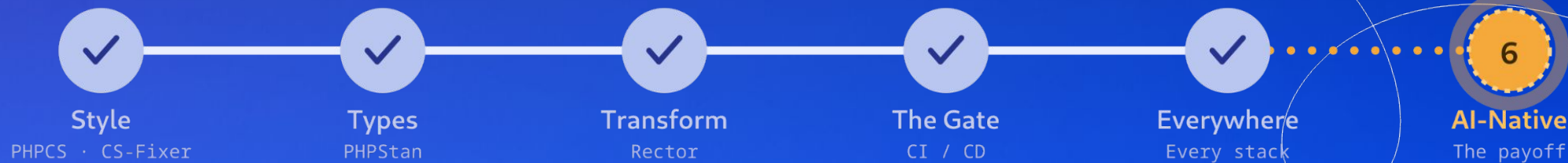
The tools change names. The pipeline doesn't.



Layer 6: AI Native Workflows

The tools aren't replaced by AI. They're what makes AI trustworthy

- Deterministic tools are not replaced by AI
- They make AI-generated code trustworthy
- Deterministic tooling = the sensory and verification layer an agent needs

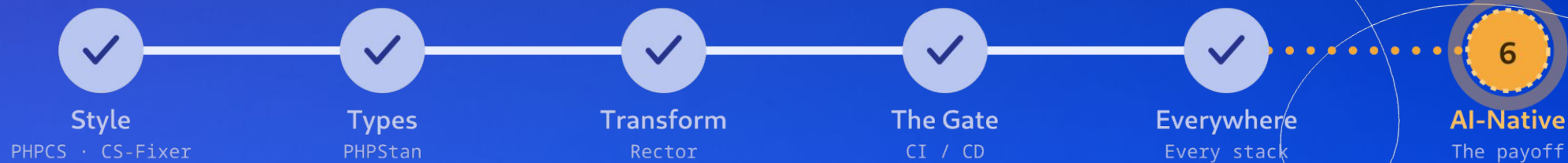


Pattern 1: AI consumes output

The tools aren't replaced by AI. They're what makes AI trustworthy

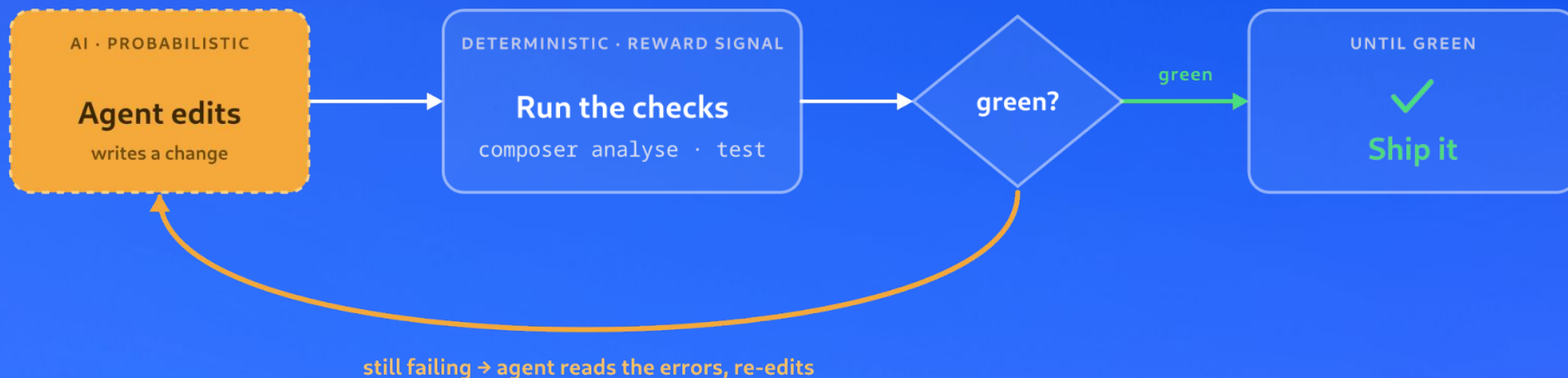


It fixes against a deterministic oracle — not a guess.

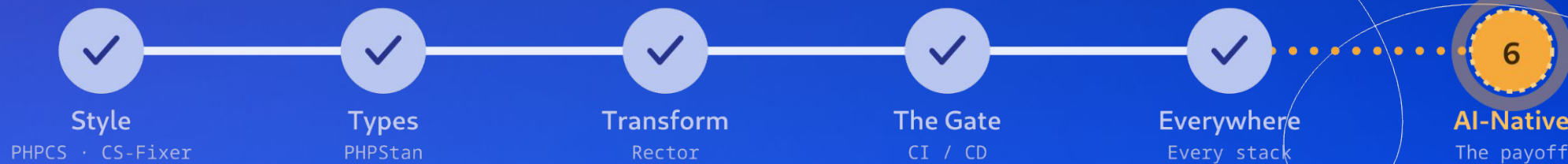


Pattern 2: The verify loop

The static analyzer is the agent's reward signal.



The checks are the agent's reward signal — it isn't done until they're green.



Pattern 3: Division of labour

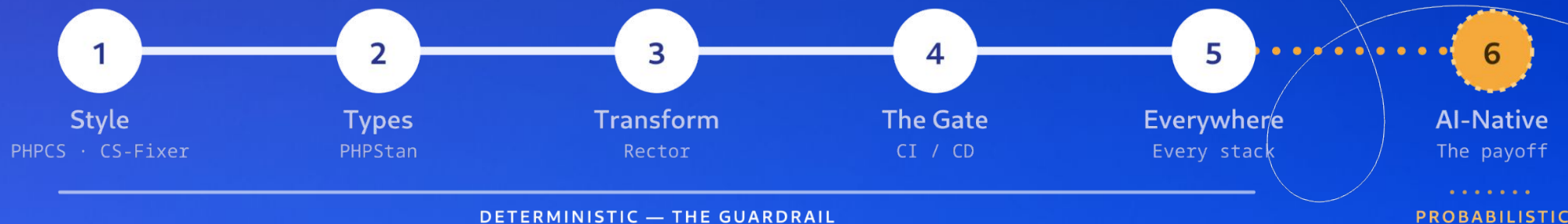
AI can even author the Rector rule

Rector is:

- Mechanical
- Repeatable
- Identical across 10k files

AI is:

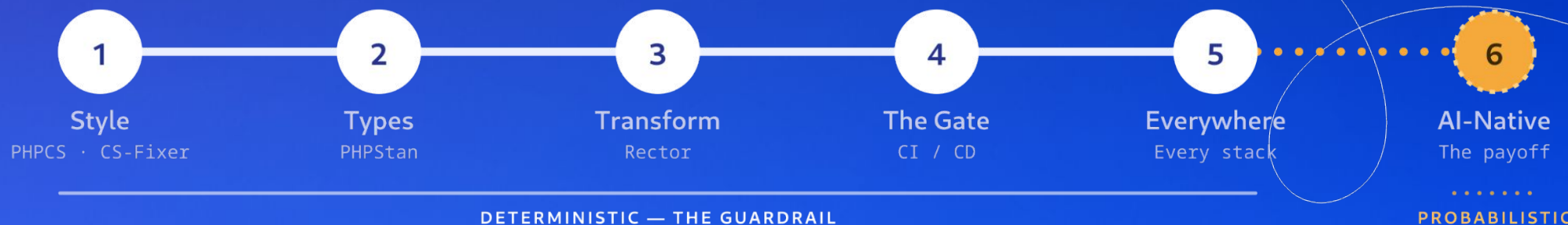
- judgment-heavy
- novel
- context-dependent



Guardrails · Ground truth · Gate.

The three things that turn "fast and fallible" into "fast and trustworthy"

AI doesn't replace static analysis - static analysis is what makes AI safe to ship



Recap

Do this week:

Engineers: **Add PHPStan level 1 + a CI gate to one repo**

Leaders: **Make the gate policy**

My fake and very, very broken example code repo:

github.com/BorislavSabev/SA-ARwithAI_example



Thank you!

Contacts:

in <https://www.linkedin.com/in/borkata/>

 <https://github.com/BorislavSabev>

@ me@borkata.com

